

BPC-PRP Project

Marek Drásal

Department of Control and Instrumentation
FEEC, Brno University of Technology
Brno, Czech Republic
256275@vutbr.cz

Pavel Dubský

Department of Control and Instrumentation
FEEC, Brno University of Technology
Brno, Czech Republic
253209@vutbr.cz

Abstract—This project focuses on the development and implementation of a maze-solving algorithm for a Robot Operating System (ROS) based robotic platform. The primary objective is to design an efficient and robust algorithm that can navigate through complex mazes, utilizing sensor data and real-time decision-making capabilities. The project will involve the integration of various sensors, such as LiDAR and camera, to perceive the environment and make informed navigation decisions.

The designed algorithm was successfully tested in maze escape competition, validating its effectiveness.

Index Terms—Robotics, project, BPC-PRP, maze-solving algorithm, ROS

I. INTRODUCTION

The main goal of this project is to develop a maze-solving algorithm for a ROS-based robotic platform. The robot will have access to various sensors, such as LiDAR and camera. Based on these sensors, the robot will control its movement via differential drive. The robot will be tested in a maze escape competition, where it will be required to navigate through a complex maze and reach the exit in the shortest possible time.

A. Robotic Platform

The robotic platform used in this project is based on Raspberry Pi 4 and Arduino Nano. The Raspberry Pi runs the ROS framework and provides sensor data to PC for processing, while the Arduino Nano handles low-level motor control and individual sensor readings [1]. The robot is equipped with a LiDAR sensor for distance measurement, a camera for visual perception of Aruco markers placed in the maze and other sensors that are not used in this implementation. The robot uses a differential drive system for movement. Simplified 3D render of the robotic platform is shown in Fig. 1.

The designed algorithm is running on the PC, which communicates with the Raspberry Pi via ROS topics. The PC processes the sensor data and sends movement commands back to the Raspberry Pi, which then relays them to the Arduino Nano for execution.

B. Typical Maze Layout

The maze is designed to be a grid of square cells, with walls separating the cells. The robot starts in a predefined cell and must navigate to the exit cell. The maze also contains Aruco markers that provide information about optimal escape path. The typical layout of the maze is shown in Fig. 2.

The maze components include:

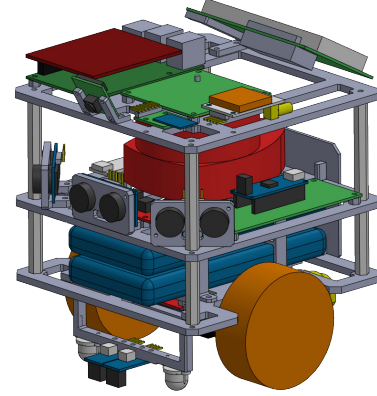


Fig. 1. 3D render of the robotic platform used in the project. [1]

- Walls: red lines
- Escape path: green line
- Start: green cell
- Treasure (bonus): blue cell
- Minotaur (penalty): red cell

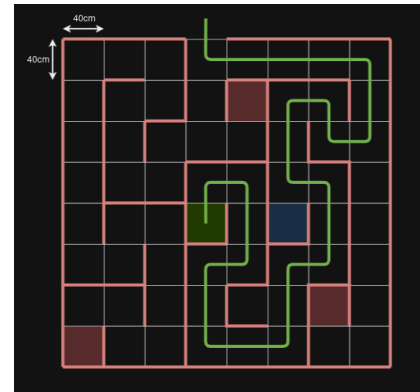


Fig. 2. Typical maze layout used in the project. [1]

II. ALGORITHM DESIGN

The algorithm is divided into three separate ROS nodes: *LiDAR navigation*, *Aruco reading* and *Motor PID Control*. The LiDAR navigation node is responsible for processing the LiDAR data and making navigation decisions based on

the detected obstacles. The Aruco reading node processes the camera data to detect Aruco markers and send the information to the navigation node. The Motor PID Control node receives movement commands from the LiDAR navigation node and controls the motors accordingly.

A. LiDAR Navigation

The navigation is divided into separate functionalities, which include correcting the robot's orientation relative to the maze, correcting position in a straight corridor and corner and crossroad detection and navigation.

Correcting the robot's orientation is achieved by converting the lidar data into a 2D occupancy grid and using the Hough transform [2] to detect the walls of the maze. The robot then adjusts its orientation to align with the detected walls. This algorithm doesn't take into account the distance to the walls, which can lead to the robot being significantly off-center in the corridor and even crashing. To correct this, the robot uses the distance measurements from the LiDAR, so when it detects that it is too close to one wall, it adjusts its trajectory to move away from that wall.

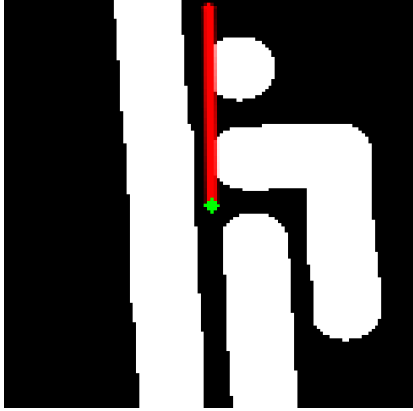


Fig. 3. Corridor image with calculated relative angle.

Corner and crossroad detection is achieved by analyzing the LiDAR data to identify the presence of walls in four directions (front, left, right, back). Based on the presence or absence of walls in these directions determined by thresholding the distances measured in the respective directions, as shown in List 1. Based on these measurements, the robot can determine whether it is in a straight corridor, at a corner, or at a crossroad. When corner is detected, the robot turns 90° in the direction of the escape path. When a crossroad is detected, the robot chooses the direction that leads to the escape path based on the latest information received from the Aruco reading node.

Listing 1. Terminal log demonstrating crossroad detection.

```
front avg: 0.99, left avg: 0.23, right avg: 0.55, back avg: 0.89
front avg: 0.99, left avg: 0.23, right avg: 0.55, back avg: 0.85
front avg: 0.99, left avg: 0.23, right avg: 0.55, back avg: 0.62
front avg: 0.99, left avg: 0.23, right avg: 0.55, back avg: 0.74
front avg: 0.99, left avg: 0.23, right avg: 0.55, back avg: 0.73
Crossroad detected!
front avg: 2.62, left avg: 1.43, right avg: 2.18, back avg: 0.19
front avg: 2.71, left avg: 1.25, right avg: 2.33, back avg: 0.16
front avg: 2.88, left avg: 1.18, right avg: 2.34, back avg: 0.16
```

B. Aruco Reading

The Aruco reading node processes the camera data to detect Aruco markers placed in the maze. Each marker contains information about the optimal escape path from that point in the maze or the location of treasure. In this implementation the treasure location is ignored. When a marker is detected, the node extracts the information and sends it to the LiDAR navigation node, which buffers it and uses it to make informed navigation decisions at the next crossroad.

For this purpose, the OpenCV library is used to detect and decode the Aruco markers.

C. Motor PID Control

The Motor PID Control node receives movement commands from the LiDAR navigation node, which are in the form of wanted speed and error angle. The node then uses a PID controller to calculate the speed for each motor to achieve the desired movement. The PID controller uses only the P component. The calculated motor speeds are then sent to the Raspberry Pi, which relays them to the Arduino Nano for execution.

III. CONCLUSION

In this project, we developed a maze-solving algorithm for a ROS-based robotic platform. The algorithm successfully navigated through a complex maze using LiDAR and camera data to make informed decisions. The robot was able to correct its orientation and position in the maze, detect corners and crossroads, and choose the optimal path based on the information from Aruco markers. The algorithm was tested in a maze escape competition, where it solved the presented maze in 1 minute and 40 seconds, which is sufficient result for the competition.

REFERENCES

- [1] BUT FEEC Robotics. *BPC-PRP*. [Online], accessed 2026-05-07. 2026. URL: <https://github.com/Robotics-BUT/BPC-PRP>.
- [2] OpenCV. *Hough Line Transform*. [Online], accessed 2026-05-07. 2025. URL: https://docs.opencv.org/3.4/d9/db0/tutorial_hough_lines.html.